

# 并行计算 II 2026 春 第一次作业报告

## 基于 OpenMP 优化直方图统计

姓名: 王梓桐

学号: 2300010750

### 1. 问题描述

#### 1.1 直方图统计算法

直方图统计 (Histogram Computation) 将连续数据离散化统计，是图像处理、数据分析和机器学习中的基础算法。给定  $N$  个浮点数  $x_0, x_1, \dots, x_{N-1}$ ，值域范围  $[\min, \max]$  和区间数  $M$ ，目标是统计每个区间 (bin) 中数据点的个数。

将值域均匀划分为  $M$  个等宽区间，宽度  $w = (\max - \min) / M$ 。对于数据点  $x_i$ ，其所属 bin 编号为：

$$\text{bin} = \left\lfloor \frac{x_i - \min}{w} \right\rfloor$$

串行算法时间复杂度  $O(N)$ ，核心代码如下：

```
double bin_width = (max_val - min_val) / M;
for (int i = 0; i < N; ++i) {
    int bin = (int)((data[i] - min_val) / bin_width);
    if (bin < 0) bin = 0;
    if (bin >= M) bin = M - 1;
    hist[bin]++;
}
```

#### 1.2 并行化的挑战

**数据竞争 (Race Condition):** 多个线程同时执行 `hist[bin]++` 时，由于该操作由“读—加—写”三步组成，并发执行会导致计数丢失。例如两个线程同时读到 `hist[k]=5`，各自加 1 后写回 6，而正确结果应

为 7。

**伪共享 (False Sharing)：** CPU 缓存以缓存行 (Cache Line，通常 64 字节) 为最小单位读写。若多个线程频繁写入同一缓存行的不同位置，会导致缓存行在各核之间反复失效和同步，造成显著性能下降。

## 2. 实现策略

本次作业实现了多种并行策略，从基本同步方案到深度微架构优化，形成渐进式优化路径。

### 2.1 基本策略 (histogram\_omp.cpp)

| 策略       | 方法                                 | 特点                           |
|----------|------------------------------------|------------------------------|
| serial   | 单线程串行                              | 基线参考                         |
| atomic   | #pragma omp atomic                 | 每次 hist[bin]++ 原子化，实现简单但竞争严重 |
| critical | #pragma omp critical               | 互斥锁保护，所有操作完全串行化              |
| private  | 线程私有数组 + 手动归约                      | 无竞争，但可能存在伪共享                 |
| padded   | 线程私有数组 + cache-line padding + 手动归约 | 消除伪共享                        |

注：OpenMP 4.5+ 的数组 reduction 语法 ( reduction(+:arr[:M]) ) 因教学集群编译器版本限制未启用（编译时自动检测）。

### 2.2 优化版本 (渐进优化路径)

| 版本          | 文件                        | 在上一版本基础上新增的优化                                       |
|-------------|---------------------------|-----------------------------------------------------|
| fast_malloc | histogram_fast_malloc.cpp | posix_memalign 64字节对齐分配 + cache-line padding + 并行归约 |
| fast_div    | histogram_fast_div.cpp    | 乘法替代除法 (预计算 inv_bin_width )                         |
| fast_ilp    | histogram_fast_ilp.cpp    | 双直方图 ILP 优化 (NREP=2, 打破 RAW 依赖链)                    |

## 2.3 伪共享实验（false\_sharing\_exp.cpp）

专用于拓展题的实验程序，支持可配置的 padding 字节数、线程数和 M 值覆盖，用于系统化测量伪共享对性能的影响。

# 3. 实验环境与方法

## 3.1 硬件平台

所有实验均在**教学集群**上完成，通过 SLURM 作业调度系统提交（Job ID: 40899，运行节点: cn09）。

| 参数      | 配置                        |
|---------|---------------------------|
| 节点数     | 1                         |
| CPU 核心数 | 16 ( --cpus-per-task=16 ) |
| 分区      | compute                   |
| 内存      | 87381 MB                  |
| 最大运行时间  | 30 分钟                     |

## 3.2 软件环境与编译选项

| 参数     | 配置                                                         |
|--------|------------------------------------------------------------|
| 编译器    | g++ -std=c++11                                             |
| 优化选项   | -O3 -march=native -flto -fno-trapping-math -fno-math-errno |
| OpenMP | -fopenmp                                                   |
| 线程绑定   | OMP_PROC_BIND=close , OMP_PLACES=cores                     |

编译选项说明：

- -O3 ：最高级别优化（循环展开、向量化、内联等）
- -march=native ：针对当前 CPU 微架构生成指令
- -flto ：链接时优化（跨编译单元内联）
- -fno-trapping-math -fno-math-errno ：放宽浮点异常语义，允许更激进的浮点优化

### 3.3 计时方法

- **计时函数：** `omp_get_wtime()`（壁钟时间，精度通常为纳秒级）
- **计时范围：** 仅包含直方图计算阶段，**不包含 I/O 读写**
- **统计方法：** 2 次 warmup 运行（预热 cache 和 TLB），随后 20 次正式运行
- **报告指标：** 均值（mean）和标准差（std）

### 3.4 输入输出组织

| 算例         | N（数据点数）            | M（bin 数） | 值域         | 文件大小          |
|------------|--------------------|----------|------------|---------------|
| input1.dat | 10,000,000 (10M)   | 256      | [0.0, 1.0) | ~80 MB (bin)  |
| input2.dat | 100,000,000 (100M) | 256      | [0.0, 1.0) | ~800 MB (bin) |

为消除 I/O 对计时的影响，文本格式输入通过 `convert_to_binary.py` 预转换为二进制格式 (.bin)。二进制格式为： `[N:i32][M:i32][min_val:f64][max_val:f64][data: Nx f64]`，可通过单次 `fread` 调用读入全部数据。

### 3.5 正确性验证

正确性验证采用**两层级设计**（见 `benchmark.sh`）：

1. **第一层（快速路径）：** 通过 `cmp -s` 将并行版本的二进制输出与已验证的串行基线输出逐字节对比。若完全一致则直接判定正确，无需额外开销。
2. **第二层（回退路径）：** 仅当二进制对比不一致时，才将输出经 `convert_to_binary.py --reverse` 转换为文本格式，调用 `check.py` 进行逐 bin 数值比对，以区分浮点舍入差异与真正的计算错误。

串行基线本身首次通过 `check.py` 验证（二进制输出转文本后与输入数据比对）。所有策略和所有线程数下均通过正确性验证（`correct=yes`）。

## 4. 基本策略性能对比

### 4.1 Input1 (N=10M, M=256) 运行时间

| 策略       | 1T (s)             | 2T (s)             | 4T (s)             | 8T (s)             | 16T (s)            |
|----------|--------------------|--------------------|--------------------|--------------------|--------------------|
| serial   | 0.02232 ± 0.000014 | —                  | —                  | —                  | —                  |
| atomic   | 0.06622 ± 0.000069 | 0.15465 ± 0.001402 | —                  | —                  | —                  |
| critical | 0.15564 ± 0.000110 | 0.91942 ± 0.002305 | —                  | —                  | —                  |
| private  | 0.02128 ± 0.000043 | 0.01081 ± 0.000011 | 0.00546 ± 0.000009 | 0.00277 ± 0.000006 | 0.00142 ± 0.000008 |
| padded   | 0.02261 ± 0.000046 | 0.01081 ± 0.000036 | 0.00546 ± 0.000007 | 0.00275 ± 0.000007 | 0.00140 ± 0.000007 |

### 4.2 Input2 (N=100M, M=256) 运行时间

| 策略       | 1T (s)             | 2T (s)             | 4T (s)             | 8T (s)             | 16T (s)            |
|----------|--------------------|--------------------|--------------------|--------------------|--------------------|
| serial   | 0.22549 ± 0.000335 | —                  | —                  | —                  | —                  |
| atomic   | 0.66242 ± 0.000343 | 1.54672 ± 0.005698 | —                  | —                  | —                  |
| critical | 1.55634 ± 0.000475 | 9.23904 ± 0.007475 | —                  | —                  | —                  |
| private  | 0.21492 ± 0.000417 | 0.10809 ± 0.000178 | 0.05452 ± 0.000061 | 0.02754 ± 0.000016 | 0.01423 ± 0.000806 |
| padded   | 0.21566 ± 0.000607 | 0.10822 ± 0.000156 | 0.05454 ± 0.000054 | 0.02747 ± 0.000014 | 0.01490 ± 0.000594 |

注：atomic 和 critical 策略因性能极差（多线程时出现严重负扩展），仅测试了 1 线程和 2 线程以节省集群计算资源。

### 4.3 加速比（相对于 serial 基线，Input2 N=100M）

| 策略       | 1T    | 2T    | 4T    | 8T    | 16T    |
|----------|-------|-------|-------|-------|--------|
| atomic   | 0.34× | 0.15× | —     | —     | —      |
| critical | 0.14× | 0.02× | —     | —     | —      |
| private  | 1.05× | 2.09× | 4.14× | 8.19× | 15.85× |
| padded   | 1.05× | 2.08× | 4.13× | 8.21× | 15.14× |

### 4.4 结果分析

#### Atomic 和 Critical 的性能灾难

**Critical**：将所有 `hist[bin]++` 操作串行化在一把全局互斥锁后面——无论线程是否访问同一个 bin，都必须排队等待。2 线程时已比串行慢 41 倍，这是因为锁的获取/释放开销远大于单次加法操作本身。

**Atomic**：在 x86 架构上通过 `lock xadd` 指令实现。虽然只在访问同一 cache line 时才会真正串行化，但 M=256 个 bin 共占 1024 字节 = 16 条 cache line。16 个线程均匀分布写入时，cache line 级别的竞争非常频繁，导致大量 cache coherence 通信。

两者的共同特点是：**线程数越多，性能越差（负扩展性）。**

#### Private 与 Padded 的对比

在本实验中（M=256），每个线程的私有直方图占  $256 \times 4 = 1024$  字节 = 恰好 16 条 cache line。理论上，若 `std::vector` 的起始地址恰好 64 字节对齐，则相邻线程的数组天然不会跨 cache line，因此 private 与 padded 性能几乎相同。

实际测量中，两者在大多数配置下性能差异确实很小（<5%），说明此处 `std::vector` 分配器碰巧产生了较好的对齐。但这种行为是**不可靠的**——稍后的伪共享实验将证明，在 M 值不同时这一巧合可能不再成立。

#### 扩展性分析

private 和 padded 策略在 1→16 线程范围内展现出接近理想的线性扩展：

| 线程数变化 | 理想加速 | private 实际加速 |
|-------|------|--------------|
| 1→2   | 2.0× | 1.99×        |

| 线程数变化 | 理想加速 | private 实际加速 |
|-------|------|--------------|
| 2→4   | 2.0× | 1.98×        |
| 4→8   | 2.0× | 1.98×        |
| 8→16  | 2.0× | 1.93×        |

16 线程时略有下降（1.93×而非2.0×），可能原因是接近内存带宽上限或操作系统调度噪声。

## 5. 优化路径：从 fast\_malloc 到 fast\_ilp

本节展示最终优化版本 histogram\_fast\_ilp 的逐步构建过程。三个版本形成递进的优化链：

fast\_malloc（内存对齐）→ fast\_div（乘法替代除法）→ fast\_ilp（指令级并行）

### 5.1 Step 1: fast\_malloc — 内存对齐与并行归约

优化内容：

- 1. 使用 posix\_memalign 确保线程私有直方图的起始地址 **64 字节对齐**（而非依赖 std::vector）
- 2. Cache-line padding: stride = padded\_M + INTS\_PER\_LINE（每线程间额外填充一个 cache line）
- 3. 归约阶段并行化：#pragma omp parallel for 将 M 个 bin 的合并分配给多线程

核心代码：

```
// 64字节对齐分配
template<typename T>
static T* aligned_alloc_array(size_t count) {
    void* ptr = nullptr;
    posix_memalign(&ptr, CACHE_LINE, count * sizeof(T));
    return static_cast<T*>(ptr);
}

// 计算阶段
const int padded_M = ((M + INTS_PER_LINE - 1) / INTS_PER_LINE) * INTS_PER_LINE;
const int stride = padded_M + INTS_PER_LINE; // +1 cache line gap
int* local_hists = aligned_alloc_array<int>(nthreads * stride);
```

性能数据 (N=100M, M=256):

| 线程数 | fast_malloc (s)    | serial (s) | 加速比    |
|-----|--------------------|------------|--------|
| 1   | 0.21012 ± 0.000495 | 0.22549    | 1.07×  |
| 2   | 0.10579 ± 0.000155 | —          | 2.13×  |
| 4   | 0.05339 ± 0.000023 | —          | 4.22×  |
| 8   | 0.02687 ± 0.000041 | —          | 8.39×  |
| 16  | 0.01389 ± 0.000734 | —          | 16.23× |

即使在单线程下，对齐分配相比 `std::vector`（padded 策略）也有 ~3% 的提升，原因是对齐的内存访问可以更高效地利用 cache line 和 prefetcher。

## 5.2 Step 2: fast\_div — 乘法替代除法

优化内容:

将热循环内的浮点除法替换为乘法:

```
// 优化前 (fast_malloc): 每次迭代一次浮点除法
double bin_width = (max_val - min_val) / M;
int bin = (int)((data[i] - min_val) / bin_width);    // fdiv: ~15-20 cycles

// 优化后 (fast_div): 循环外预计算倒数，循环内仅需乘法
double inv_bin_width = (double)M / (max_val - min_val); // 仅计算一次
int bin = (int)((data[i] - min_val) * inv_bin_width);    // fmul: ~4-5 cycles
```

**原理:** 在现代 x86 CPU 上，浮点除法（`fdiv / divsd`）的延迟约为 15-20 个时钟周期，而浮点乘法（`fmul / mulsd`）仅需 4-5 个周期。由于 `bin_width` 在整个循环中不变，可以在循环外计算其倒数 `inv_bin_width = M / (max - min)`，将循环内的除法转化为乘法。

性能数据 (N=100M, M=256):

| 线程数 | fast_div (s)       | fast_malloc (s) | 相对提升   |
|-----|--------------------|-----------------|--------|
| 1   | 0.17265 ± 0.000551 | 0.21012         | +21.7% |
| 2   | 0.08764 ± 0.000448 | 0.10579         | +20.7% |
| 4   | 0.04420 ± 0.000097 | 0.05339         | +20.8% |

| 线程数 | fast_div (s)       | fast_malloc (s) | 相对提升   |
|-----|--------------------|-----------------|--------|
| 8   | 0.02279 ± 0.000057 | 0.02687         | +17.9% |
| 16  | 0.01195 ± 0.000829 | 0.01389         | +16.2% |

单线程下获得 **21.7% 的性能提升**，这完全来自消除了每次迭代的浮点除法。多线程时提升比例略有减小，是因为在高并行度下内存带宽逐渐成为瓶颈，计算优化的相对收益被稀释。

## 5.3 Step 3: fast\_ilp — 指令级并行

### 什么是 ILP (Instruction-Level Parallelism)

**指令级并行 (ILP)** 是现代超标量处理器利用流水线和乱序执行能力，在同一时钟周期内并行处理多条指令的技术。CPU 可以同时执行多条互不依赖的指令，但如果存在数据依赖（如 Read-After-Write），则后续指令必须等待前一条完成。

在直方图计算中，核心操作 `hist[bin]++` 形成一条依赖链：

- 1. **读取** `hist[bin]` 的当前值 (Load)
- 2. **加 1** (Add)
- 3. **写回** 到 `hist[bin]` (Store)

如果连续两次操作碰巧写入同一个 bin（或 CPU 无法在编译/执行时证明它们写的是不同地址），第二次操作必须等第一次的写回完成，形成 **RAW (Read-After-Write) 依赖链**，限制了流水线效率。

### 优化方法

为打破 RAW 依赖链，每个线程维护 **2 份直方图副本** (`hist0` 和 `hist1`)，交替处理连续数据点：

```
static const int NREP = 2; // dual histograms for ILP
int* hist0 = local_hists + (tid * NREP + 0) * stride;
int* hist1 = local_hists + (tid * NREP + 1) * stride;

#pragma omp for schedule(static)
for (int i = 0; i < N - 1; i += 2) {
    int bin0 = (int)((data[i] - dmin) * inv_bin_width);
    hist0[bin0]++; // 写入 hist0

    int bin1 = (int)((data[i+1] - dmin) * inv_bin_width);
    hist1[bin1]++; // 写入 hist1 - 与 hist0 无依赖!
}
```

由于 hist0 和 hist1 是完全独立的内存区域，CPU 流水线可以在 hist0[bin0]++ 的写回尚未完成时，立即开始执行 hist1[bin1]++ 的加载和计算。这有效地将单条依赖链拆分为两条独立的短链，提高了流水线利用率。

性能数据（N=100M, M=256）

| 线程数 | fast_ilp (s)       | fast_div (s) | 相对提升         | 总加速比 (vs serial) |
|-----|--------------------|--------------|--------------|------------------|
| 1   | 0.16470 ± 0.000846 | 0.17265      | <b>+4.8%</b> | 1.37×            |
| 2   | 0.08328 ± 0.000258 | 0.08764      | +5.2%        | 2.71×            |
| 4   | 0.04204 ± 0.000130 | 0.04420      | +5.1%        | 5.36×            |
| 8   | 0.02138 ± 0.000045 | 0.02279      | +6.6%        | 10.55×           |
| 16  | 0.01126 ± 0.000679 | 0.01195      | +6.1%        | <b>20.02×</b>    |

ILP 优化在所有线程数下都提供了稳定的 5-7% 额外性能收益。虽然比例不大，但这是在已经高度优化基础上的进一步提升，且代码改动极小。

5.4 优化路径总结

| 版本              | 1T (s)        | 16T (s)        | 16T 加速比       | 相对上一步提升 (1T) |
|-----------------|---------------|----------------|---------------|--------------|
| serial (基线)     | 0.2255        | —              | 1.00×         | —            |
| fast_malloc     | 0.2101        | 0.01389        | 16.23×        | 对齐内存 +7%     |
| fast_div        | 0.1727        | 0.01195        | 18.87×        | 乘法替代除法 +22%  |
| <b>fast_ilp</b> | <b>0.1647</b> | <b>0.01126</b> | <b>20.02×</b> | ILP +5%      |

三步优化累计使 16 线程性能从 16.23× 提升到 **20.02×**（相当于理论上超越了线程数的加速），这得益于单线程性能本身也从 0.2255s 优化到了 0.1647s（串行计算量减少了 27%）。

6. 伪共享实验（拓展题）

6.1 实验设计

为系统化研究伪共享（False Sharing）对并行直方图性能的影响，设计了三维参数扫描实验：

| 参数           | 取值                                 |
|--------------|------------------------------------|
| M (bin 数)    | 8, 33, 128                         |
| Padding (字节) | 0, 4, 8, 16, 32, 64, 128, 256, 512 |
| 线程数          | 4, 16                              |
| N            | 100,000,000 (固定)                   |
| 统计方法         | 2 warmup + 20 runs                 |

M 值选取的理由：

- **M=8**：每线程数组 32 字节 < 1 cache line，极端伪共享场景
- **M=33**：每线程数组 132 字节， $\text{spill} = 132 \% 64 = 4$  字节（边界错位）
- **M=128**：每线程数组 512 字节， $512 \% 64 = 0$ （**理论对齐**）

## 6.2 实验结果

**M=8（每线程 32 字节，严重伪共享）**

| Padding (B) | 4线程 (s)                  | 16线程 (s)                 |
|-------------|--------------------------|--------------------------|
| 0           | 0.2668 ± 0.000283        | 0.1303 ± 0.000106        |
| 4           | 0.2599 ± 0.007146        | 0.0992 ± 0.000067        |
| 8           | 0.2061 ± 0.000125        | 0.0835 ± 0.000102        |
| 16          | 0.1483 ± 0.000098        | 0.0563 ± 0.000140        |
| 32          | 0.2263 ± 0.000152        | 0.0172 ± 0.000733        |
| <b>64</b>   | <b>0.0566 ± 0.000037</b> | <b>0.0189 ± 0.000678</b> |
| 128         | 0.0542 ± 0.000047        | 0.0146 ± 0.000783        |
| 256         | 0.0527 ± 0.000164        | 0.0138 ± 0.001039        |
| 512         | 0.0527 ± 0.000114        | 0.0138 ± 0.001025        |

M=33（每线程 132 字节，边界错位）

| Padding (B) | 4线程 (s)           | 16线程 (s)          |
|-------------|-------------------|-------------------|
| 0           | 0.1571 ± 0.000085 | 0.0715 ± 0.000074 |
| 4           | 0.1669 ± 0.000099 | 0.0648 ± 0.000108 |
| 8           | 0.1856 ± 0.000126 | 0.0559 ± 0.000180 |
| 16          | 0.1385 ± 0.000076 | 0.0403 ± 0.000048 |
| 32          | 0.1027 ± 0.000046 | 0.0314 ± 0.000051 |
| 64          | 0.0575 ± 0.000057 | 0.0160 ± 0.000667 |
| 128         | 0.0544 ± 0.000026 | 0.0142 ± 0.001066 |
| 256         | 0.0544 ± 0.000115 | 0.0142 ± 0.001002 |
| 512         | 0.0537 ± 0.000121 | 0.0142 ± 0.001043 |

M=128（每线程 512 字节，理论对齐）

| Padding (B) | 4线程 (s)           | 16线程 (s)          |
|-------------|-------------------|-------------------|
| 0           | 0.1221 ± 0.000516 | 0.0339 ± 0.000076 |
| 4           | 0.0929 ± 0.000554 | 0.0345 ± 0.000167 |
| 8           | 0.0923 ± 0.002405 | 0.0301 ± 0.000055 |
| 16          | 0.0813 ± 0.000110 | 0.0263 ± 0.000127 |
| 32          | 0.0554 ± 0.000050 | 0.0203 ± 0.000746 |
| 64          | 0.0550 ± 0.000074 | 0.0145 ± 0.000961 |
| 128         | 0.0542 ± 0.000051 | 0.0143 ± 0.001049 |
| 256         | 0.0537 ± 0.000147 | 0.0139 ± 0.000806 |
| 512         | 0.0537 ± 0.000090 | 0.0139 ± 0.000775 |

6.3 分析一：M 大小对伪共享惩罚的影响

以 16 线程为例，对比无 padding 和最优 padding 的性能：

| M   | 每线程数组 (B) | pad=0 (s) | 最优 pad (s) | 加速比  |
|-----|-----------|-----------|------------|------|
| 8   | 32        | 0.1303    | 0.0138     | 9.5× |
| 33  | 132       | 0.0715    | 0.0142     | 5.0× |
| 128 | 512       | 0.0339    | 0.0139     | 2.4× |

规律：M 越小，伪共享的惩罚越大。这是因为：

- 当 M 很小时（如 M=8），每线程私有数组不足一个 cache line，相邻线程的数据**必然**落在同一 cache line 中，每次写操作都会触发 cache line invalidation
- 当 M 增大时，虽然线程边界处仍可能存在共享 cache line，但该 cache line 中的 bin 被访问的概率 = 1/M，因此伪共享触发频率大幅降低

## 6.4 分析二：Padding 大小的影响

以 M=33、16 线程为例分析 padding 的渐进效果：

| Padding (B) | 时间 (s)        | 相对 pad=0    | 分析                           |
|-------------|---------------|-------------|------------------------------|
| 0           | 0.0715        | 基线          | 完全暴露的伪共享                     |
| 4           | 0.0648        | -9%         | 推开了 4 字节但仍在同一 cache line     |
| 8           | 0.0559        | -22%        | 同上                           |
| 16          | 0.0403        | -44%        | 部分缓解                         |
| 32          | 0.0314        | -56%        | 可能跨越了部分 cache line 边界        |
| <b>64</b>   | <b>0.0160</b> | <b>-78%</b> | <b>恰好一个 cache line → 质变！</b> |
| 128         | 0.0142        | -80%        | 边际改善                         |
| 256         | 0.0142        | -80%        | 收敛                           |

**关键发现：64 字节是伪共享消除的关键阈值。**因为 CPU cache line 大小为 64 字节，只有当相邻线程的数据间隔达到整个 cache line 宽度时，才能彻底消除 cache line 共享。部分 padding (<64B) 虽有改善但无法根治问题。

## 6.5 分析三：线程数对伪共享的影响

以 M=8 为例对比 4 线程和 16 线程：

| Padding         | 4线程 (s) | 16线程 (s) | 16T/4T |
|-----------------|---------|----------|--------|
| 0 (伪共享)         | 0.2668  | 0.1303   | 0.49   |
| 64 (消除)         | 0.0566  | 0.0189   | 0.33   |
| 比值 pad=0/pad=64 | 4.7×    | 6.9×     | —      |

分析：

- **消除伪共享后** (pad=64)，4T→16T 的加速比 =  $0.0566/0.0189 = 3.0\times$  (理想值 4.0×)，说明存在其他瓶颈 (如内存带宽)
- **有伪共享时** (pad=0)，4T→16T 的加速比 =  $0.2668/0.1303 = 2.0\times$  (远低于理想)，说明更多线程意味着更多核参与 cache coherence 协议，invalidation 通信开销急剧增加
- 伪共享惩罚从 4 线程的 4.7× 增长到 16 线程的 **6.9×**，证实线程数越多，伪共享造成的 cache 一致性开销越大

## 6.6 分析四：M=128 的特殊情况 — 为何理论对齐仍有伪共享

这是本实验最具启发性的发现。

**理论预期：**M=128 时，每线程私有数组占  $128 \times 4 = 512$  字节 = 恰好 8 条 cache line。由于 512 是 64 的整数倍，如果内存起始地址对齐，相邻线程的数组**不会**跨越 cache line 边界，**不应出现伪共享**。

**实际测量：**pad=0 时耗时 0.0339s，而 pad≥64B 后降至 0.0139s — 仍有 **2.4×** 的性能差距！

**根本原因：**std::vector 不保证 64 字节对齐

false\_sharing\_exp.cpp 中使用 `vector<int> local_hists(nthreads * stride, 0)` 分配内存。标准 C++ 分配器 (new / malloc) 通常只保证 8 字节或 16 字节对齐。假设 std::vector 内部缓冲区的起始地址为 base，且  $base \% 64 = k$  (其中  $0 < k < 64$ )，那么：

- 线程 0 的数组起始于  $base + 0$  (偏移 k 字节)
- 线程 1 的数组起始于  $base + 512$  (偏移  $(k + 512) \% 64 = k$  字节)
- 线程 T 的数组起始于  $base + T \times 512$  (偏移 k 字节)

虽然相邻线程数组间距是 cache line 的整数倍 ( $512/64=8$ )，但如果  $k \neq 0$ ，则**每个线程的数组都跨越了一个 cache line 边界**。具体来说：

```

Cache Line:  |----- 64B -----|----- 64B -----|
Thread T:      [===== 512B =====]
Thread T+1:      [===== 512B =====]
                ^

```

这条 cache line 被两个线程共享！

如果起始地址偏移了 k 字节，则线程 T 的最后 k 字节和线程 T+1 的前 (64-k) 字节落在同一条 cache line 上，当两个线程分别写入这些位置时就会触发伪共享。

## 回扣 histogram\_fast\_malloc 的优化动机

这一发现恰好解释了为什么 histogram\_fast\_malloc.cpp 要使用 posix\_memalign(&ptr, 64, ...) 而非 std::vector：

```

// histogram_fast_malloc.cpp 的做法
int* local_hists = aligned_alloc_array<int>(nthreads * stride);
// 等价于：posix_memalign(&ptr, 64, nthreads * stride * sizeof(int));

```

posix\_memalign **保证**返回的指针是 64 字节对齐的（即 k=0）。配合 cache-line padding 的 stride，彻底消除了伪共享的可能性。这就是为什么即使在 M=256（理论对齐）时，fast\_malloc 也能比 padded（基于 std::vector）更稳定、更快。

**总结：**消除伪共享需要两个条件同时满足：

1. **Stride 对齐：**相邻线程间距为 cache line 的整数倍（padding）
2. **起始地址对齐：**首线程数组起始于 cache line 边界（posix\_memalign）

缺少任何一个条件都可能残留伪共享，M=128 的实验完美地展示了这一点。

# 7. 项目结构与复现指南

## 7.1 文件结构

```
parco2026_HW1/
├── C++ 源代码
│   ├── serial.cpp                # 串行基线实现
│   ├── histogram_omp.cpp         # 多策略 OpenMP 实现 (serial/atomic/critical/private/padded)
│   ├── histogram_fast_malloc.cpp # 优化 Step 1: 内存对齐 + padding
│   ├── histogram_fast_div.cpp    # 优化 Step 2: + 乘法替代除法
│   ├── histogram_fast_ilp.cpp    # 优化 Step 3: + 双直方图 ILP
│   └── false_sharing_exp.cpp     # 伪共享实验 (可配置 padding/M/线程数)
├── 构建与运行脚本
│   ├── Makefile                 # 构建系统 (含 OpenMP 4.5+ 自动检测)
│   ├── run.slurm                # SLURM 集群提交脚本
│   ├── run_local.sh             # 本地实验运行脚本
│   ├── benchmark.sh             # 策略对比基准测试
│   ├── false_sharing_bench.sh   # 伪共享 padding 扫描实验
│   └── cleanup.sh               # 清理中间文件
├── Python 工具
│   ├── gen_input.py             # 生成测试输入数据
│   ├── convert_to_binary.py     # 文本格式 → 二进制格式转换
│   └── check.py                 # 正确性验证 (逐 bin 比对)
├── 实验结果
│   ├── job_40899.out            # SLURM 作业完整输出日志
│   └── results_slurm/
│       ├── benchmark_20260510_130118.csv # 策略对比数据 (CSV)
│       └── fs_padding_sweep_20260510_131218.csv # 伪共享实验数据 (CSV)
├── 文档
│   ├── HW1_report.md           # 本报告
│   └── HW1_report.pdf          # 本报告
└── .gitignore                  # Git 忽略规则
```

## 7.2 各文件说明

| 文件                        | 用途                               |
|---------------------------|----------------------------------|
| serial.cpp                | 串行参考实现，使用 <chrono> 计时，不依赖 OpenMP |
| histogram_omp.cpp         | 统一的多策略实现，通过命令行参数选择策略             |
| histogram_fast_malloc.cpp | 优化路径的第一步：对齐内存 + padding + 并行归约   |
| histogram_fast_div.cpp    | 第二步：在 Step 1 基础上用倒数乘法替代热循环除法     |
| histogram_fast_ilp.cpp    | 第三步：在 Step 2 基础上实现双直方图 ILP       |
| false_sharing_exp.cpp     | 可配置 padding/M/线程数的伪共享测量程序        |
| Makefile                  | 自动检测 OpenMP 4.5+ 支持，构建所有目标       |
| run.slurm                 | 一键提交到教学集群运行全部实验                  |
| benchmark.sh              | 策略 × 线程数 × 输入 全扫描，输出 CSV         |
| false_sharing_bench.sh    | M × padding × 线程数 参数扫描           |
| gen_input.py              | 生成均匀分布的浮点测试数据                    |
| convert_to_binary.py      | 将文本输入转为二进制格式以加速读取                |
| check.py                  | 验证并行计算结果与串行结果一致                  |

## 7.3 复现实验

### 方式一：教学集群（推荐）

```
cd parco2026_HW1/  
sbatch run.slurm
```

run.slurm 会自动完成以下步骤：

1. 生成输入数据 ( gen\_input.py )
2. 编译所有目标 ( make all )
3. 运行策略对比实验 ( benchmark.sh )
4. 运行伪共享实验 ( false\_sharing\_bench.sh )
5. 将所有结果保存到 results\_slurm/ 目录
6. 完成后自动清理中间文件

## 方式二：本地运行

```
cd parco2026_HW1/
chmod +x run_local.sh benchmark.sh false_sharing_bench.sh cleanup.sh
./run_local.sh results
```

## 单独运行某个策略

```
make all
python3 gen_input.py
python3 convert_to_binary.py input1.dat input1.bin

# 运行多策略版本
OMP_NUM_THREADS=16 ./histogram_omp private input1.bin output.bin

# 运行最优化版本
OMP_NUM_THREADS=16 ./histogram_fast_ilp input1.bin output.bin

# 验证正确性
python3 convert_to_binary.py --reverse output.bin output.dat
python3 check.py input1.dat output.dat
```

# 8. 总结

## 8.1 核心成果

| 指标      | 结果                           |
|---------|------------------------------|
| 最优策略    | histogram_fast_ilp           |
| 最大加速比   | <b>20.02×</b> （16线程, N=100M） |
| 所有策略正确性 | 全部通过 ✓                       |

## 8.2 关键发现

- 同步策略的选择至关重要：**atomic 和 critical 导致严重负扩展（性能随线程数下降），在直方图等高频更新场景中应完全避免。线程私有数组 + 归约是正确的并行化模式。

## 2. 三步优化各有贡献：

- 内存对齐：确保 cache line 级别的高效访问（+7%）
- 乘法替代除法：消除热循环中最昂贵的指令（+22%）
- ILP 双直方图：利用微架构流水线能力（+5%）

## 3. 伪共享消除需要双重保障：

- 仅 padding（如 padded 策略）不够——分配器可能不对齐起始地址
- 需要 `posix_memalign` + padding 同时使用才能彻底消除